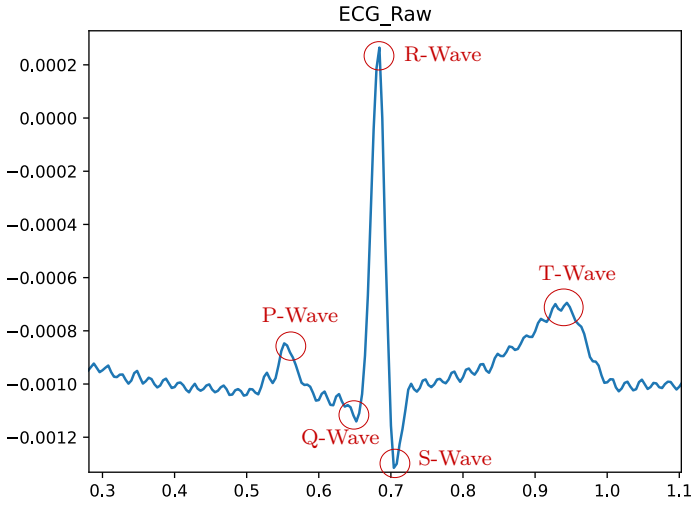
Digital Signal Processing
ECG FIR Filtering Assignment

2785209M & 2664680S

1 Plotting the ECG Signal

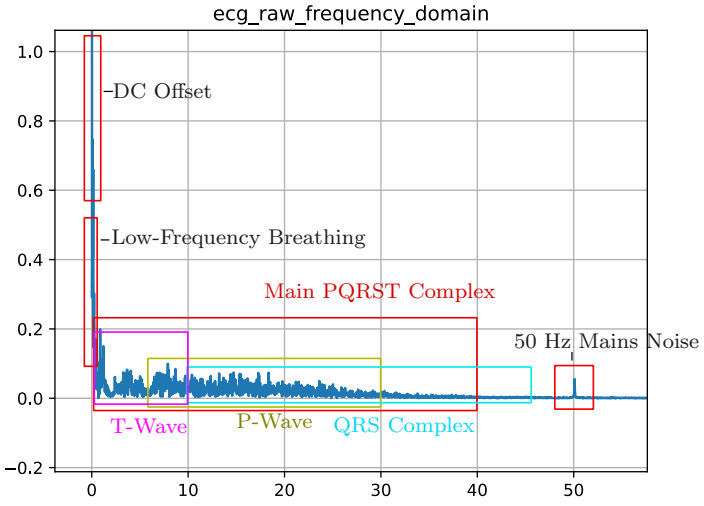
The experiment began by taking an ECG recording and plotting it in Python using Matplotlib. The electrodes were attached just above the hip and on the front of the shoulder.

1.1 Structure of PQRST Complex



It is important that we preserve the frequencies used to construct the PQRST complex, otherwise we would be destroying important data from the ECG.

1.2 Sources of Noise in the ECG



The main sources of noise in an ECG are 50Hz mains interference, Breathing, and random noise caused by movement. There is also a DC component which shifts the entire signal by a certain amount, caused by potential difference formed between the electrodes and the skin.
The human heart rate can go as low as 0.5Hz at complete rest. Considering this, the filter should remove frequencies below 0.5Hz (This includes DC shift and Breathing) and above 45Hz (Mains noise and other high frequency interference). Although this leaves in random noise caused by movement, the core frequencies of the PQRST complex must be retained for the signal to remain accurate for monitoring purposes.

2 Generating the FIR Coefficients

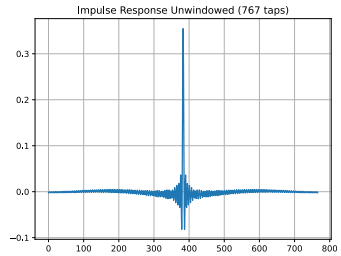
2.1 Stop-Band Filter

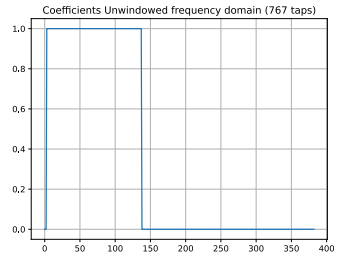
A stop-band filter was implemented in this code by utilizing an all-pass filter and setting specific indices within it to zero before converting it into the time domain.
To create the all-pass filter, first the number of coefficients needed for the windowing function to work had to be calculated. For a Kaiser Window, this changes based on the desired stopband attenuation and transition width.

number of taps:

$$N = \left\lfloor \frac{A - 8}{2.285 \cdot 2\pi \left(\frac{\Delta f}{f_s}\right)} \right\rfloor \tag{1}$$

An ideal all-pass filter (H) was defined in the frequency domain by creating an array of all ones with length `num_taps`. This all-pass filter was then modified in a for loop to remove the specific stop-band frequencies, by setting the indices which correspond to those frequencies to zero - Not including the DC component. Finally, an Inverse Fourier Transform is done on the resulting array after all of the stop bands have been iterated over. This results in an unwindowed stop-band filter which would produce ringing due to the Gibbs phenomenon if applied raw. This can be seen in the impulse response of the filter as the side lobes aren't able to settle to zero before they get truncated:





To fix this, the coefficients must be multiplied with a windowing function.

This frequency sampling approach approximates the analytical method of calculating the ideal time-domain band-pass impulse response using sinc functions, and subtracting it from a unit impulse ($\delta[n]$) via the principle of superposition:

$$h_{\text{band}}[n] = 2f_2 \,\text{sinc}(2f_2 n) - 2f_1 \,\text{sinc}(2f_1 n) \tag{2}$$

$$h_{\text{stop}}[n] = \delta[n] - \sum h_{\text{band}}[n] \tag{3}$$

(Note: Calculating the IDFT analytically via sinc functions is less convenient for multi-band filters than using `np.fft.ifft()`, and manual $\frac{\sin(x)}{x}$ evaluation requires handling the division-by-zero edge case at $n = 0$.)

2.2 The Kaiser Window

The Kaiser window was chosen for its flexibility and simplicity. It gives greater flexibility than other windowing functions, since the user has control over both the size of the transition window and the stopband attenuation. This allows you to find a precise balance between a narrow central peak (sharper cutoff) and flatter sidelobes (better noise rejection) in the impulse response. With specific stopband attenuations, it is also possible for the Kaiser window to mimic other windowing functions. Furthermore, the Kaiser filter has straightforward formulas to calculate beta (the stopband attenuation parameter) and `num_taps` (the number of coefficients needed for the filter), allowing it to be practically calculated almost entirely in code without external libraries. The array n represents discrete time indices from $-m$ to m. Evaluating functions over this symmetric time vector is what gives the filter linear phase and constant group delay. The formulae used in the code are as follows:

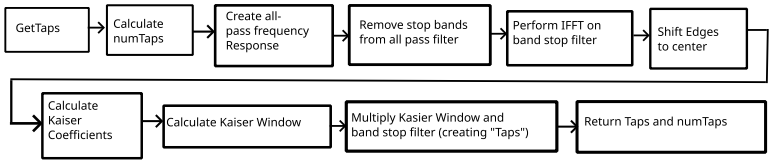
beta parameter:

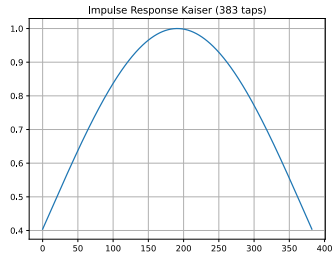
$$\beta = \begin{cases} 0.1102(A-8.7) & \text{if } A > 50 \\ 0.5842(A-21)^{0.4} + 0.07886(A-21) & \text{if } 21 \leq A \leq 50 \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

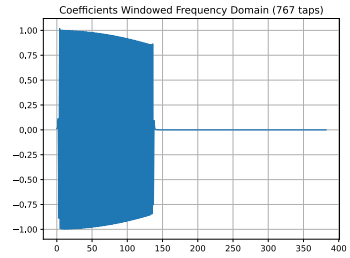
kaiser window:

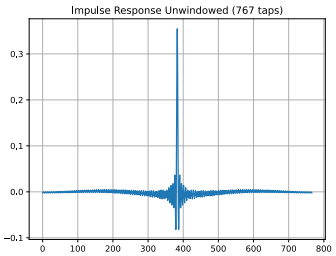
$$w[n] = \frac{I_0\left(\beta\sqrt{1-\left(\frac{n}{m}\right)^2}\right)}{I_0(\beta)} \quad (5)$$

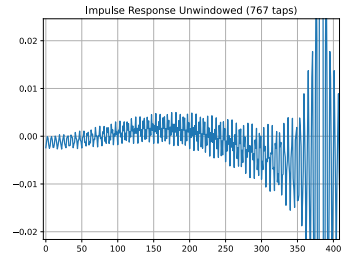
The FIR filter is then multiplied by the Kaiser window, to create the final array of coefficients, producing this impulse response in the time domain:

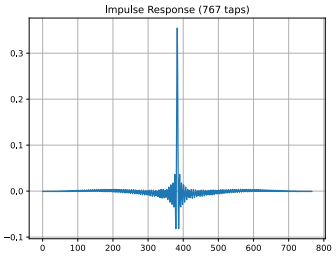


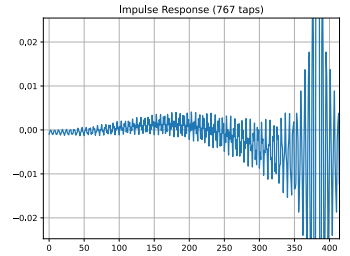












As you can see, this impulse response has a large, narrow central peak and sidelobes that gradually decay to zero as they approach tap 0 and tap 383. This shows that our windowing function has successfully removed ringing due to the Gibbs Phenomenon.

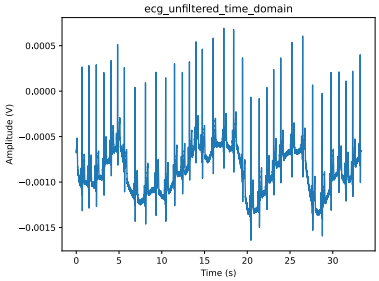
These coefficients can now be passed into the `fir_filter` class to simulate realtime filtering.

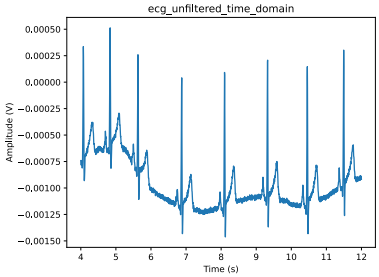
3 Simulation of Real-time Filtering

In order to simulate realtime filtering of the data, a python class was created to represent the FIR class. This class takes the taps (calculated from the filter design function) as it's sole argument. Additionally, the class has 3 data members; coefficients is an array of the filter's taps; num_taps is the length of coefficients; And delay_line is a double ended queue with a maximum length of num_taps. A double ended queue was used as the data structure for the delay line because, when given a maximum length, it acts similarly to a ring buffer where adding a new value to a full queue causes one value to fall off the other end. This is an efficient implementation, because it does not require manual shifting of the delay line.
The function member used for simulating realtime processing is simply dofilter. This takes a scalar input of the newest sample in the data, appends it to the delay line and then loops through the delay line to calculate a cumulative sum of delay_line[i] * coefficients[i]. It returns the result of this summation. To simulate realtime input to the filter in python, a for loop iterates through the ecg data and passes it sample by sample to the FIR filter for processing.

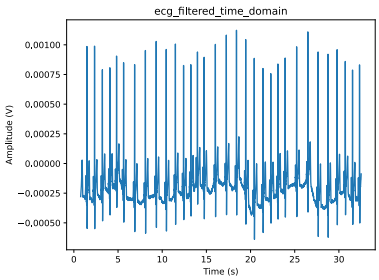
3.1 Filter Design

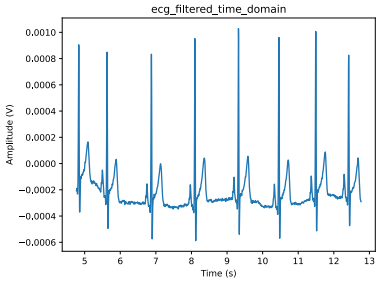
The raw ECG signal can be seen below:





To filter our ECG signal, we used a bandpass filter with a lower cutoff frequency of 1Hz and an upper cutoff frequency of 45Hz. Although this does not completely remove low frequency drift from the signal, it was found that increasing the lower cut off frequency further would degrade the quality of the PQRST wave and so 1Hz was chosen as the optimal value to balance out these trade-offs. It can also be seen that mains noise has been completely removed from the filtered signal:





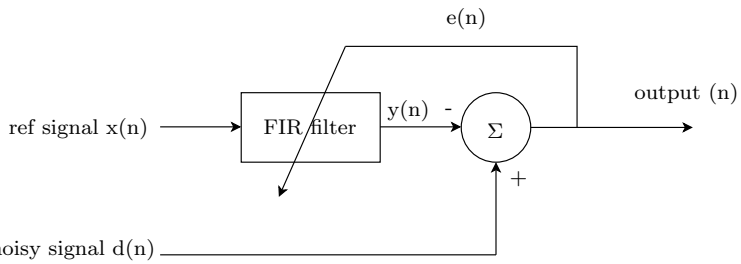
4 Adaptive LMS Filter

The doFilterAdaptive function member extends the FIR filter class for adaptive filtering. The arguments to this function are signal (x(n)), noise (d(n)), and learningRate. The signal argument is the signal that we are trying to remove from our ECG. The noise argument is our noisy ECG signal, and learningRate is the learning rate/step size of our adaptive algorithm. The function initially calls the doFilter function on the signal for the forward pass before updating the weights of the filter using a least mean squares method:

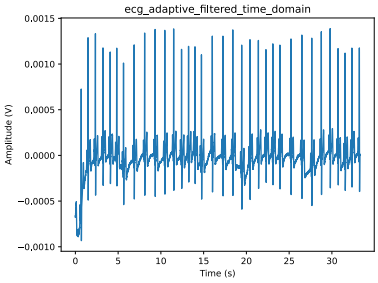
$$\text{error} = \text{noise} - \text{filtered_signal} \quad (6)$$

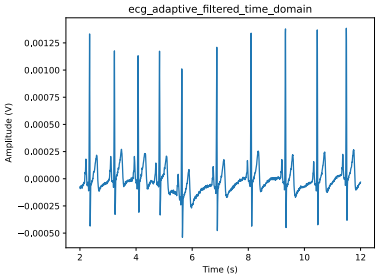
$$\text{coefficients}[i] = \text{coefficients}[i] + \text{learningRate} \cdot \text{error} \cdot \text{delay_line}[i] \quad (7)$$

In this way, the cofficients are updated until any noise, x(n), that is correlated with d(n) is removed from the signal. Our signal input is a 50Hz sin wave plus a DC shift of 0.3. The value of the DC shift was found experimentally with the goal of removing the DC shift from the signal without degrading the quality of the PQRST wave. The learning rate was also found experimentally such that the filter coefficients converge quickly without exploding. A dataflow diagram can be seen below:



As well as the output from the adaptive filter:



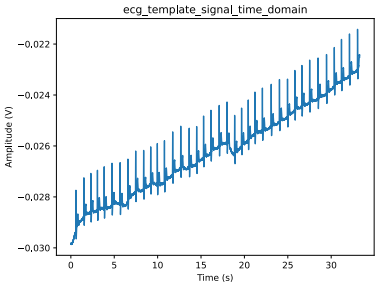


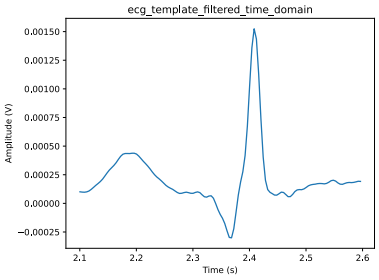
It can be seen that although it takes a few seconds for the filter to converge, the 50Hz mains noise is completely removed from the signal thereafter. Additionally, the DC shift is also removed from the signal as well as the majority of the low frequency drift. However, the DC term was also found to slightly decrease the strength of the T wave as well as introducting a momentary trough immediately after it.

Compared to the FIR filter the adaptive filter is able to perfectly remove the 50Hz mains noise and DC from the signal, whereas the FIR filter is only able to attenuate them. Additionally, the ECG is not distorted at all by the adaptive filter, whereas a narrow bandstop FIR filter will always distort the ECG signal to some degree. The benefits of the adaptive fiter, however, come at the cost of an initial convergence time and a more computationally complex implementation. In both cases, removal of the low frequency drift comes at the cost of some distortion of the PQRST wave, but this distortion was more pronounced with the adaptive filter.

5 Heartbeat Detection

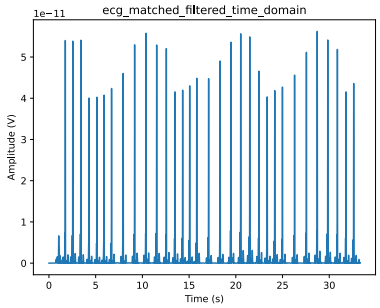
Heartbeat was detected using a matched filter. This process requires a heartbeat template which can then be correlated with the ECG signal to find the location of the heartbeats. The other team member's ECG signal was used to create this template. The ECG signal was first filtered using the FIR filter to remove low frequency drift and mains noise. The lower frequency cut off was increased from 1Hz to 3Hz to remove more of the low frequency drift from the ECG signal and the template signal. Although this diminishes the clarity of the PQRST wave, it was found to be more important for the matched filter to be working with signals without DC and low frequency drift than it was to retain a clean PQRST wave. A PQRST wave was then extracted and reversed to use as the filter coefficients of the matched filter. The PQRST template has to be reversed in order for the convolution of the FIR filter to act as a correlation operation between the template and the ECG signal. The selected template as well as the unfiltered data from which it was taken are shown below:

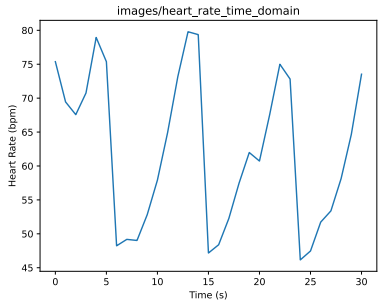




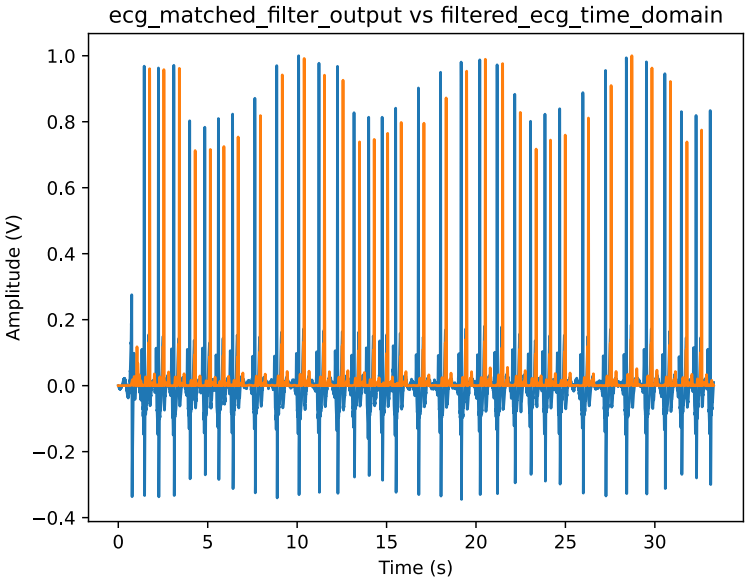
The constant linear drift of the ECG signal we needed to obtain the template from can be seen here. This impacted the quality of the template that we were able to extract, but it was still sufficient for the purposes of heartbeat detection.

The output from the matched filter is then squared to amplify the SNR. A threshold is set to detect the peaks of the output signal which correspond to heartbeats, and the heartrate is calculated as the inverse of the time between detected peaks. Peaks were detected using a threshold value of 3×10^{-11}V and a heuristic is applied by setting a cooldown time between detected peaks which limits the heartrate to 220bpm. This helps to avoid any false positives. The output from the matched filter as well as the heartrate detected were as follows:



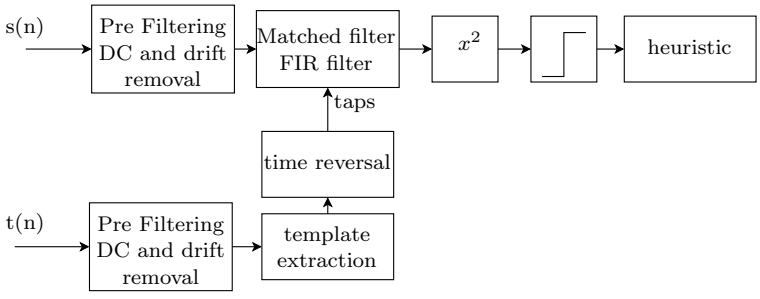


By normalising the signals to make them easier to compare, it can be seen that the peaks of the matched filter output correspond to the R peaks of the ECG signal, meaning we have succesfully detected the heartbeats present in the ECG signal.



The delay seen is simply the time it takes for the matched filter to process the signal (the group delay), and does not affect the accuracy of the heartbeat detection.

A dataflow diagram of the process can be seen below:



6 Appendix

6.1 ECG Plotting Function

```
import numpy as np
from pathlib import Path
import matplotlib.pyplot as plt

IMAGES = Path("images")
IMAGES.mkdir(parents = True, exist_ok = True)

file_name = "James_ecg_1.tsv"
tsv = np.loadtxt(file_name, delimiter='\t')
ecg_signal = tsv[:,8].copy()

sample_rate = 250
num_samples = len(ecg_signal)
measurement_time = num_samples / sample_rate
t = np.linspace(0, measurement_time, num_samples)

plt.figure()
plt.title("time_domain_ecg")
plt.plot(t,ecg_signal)
plt.savefig(IMAGES / f"time_domain_{file_name}.svg")
plt.show()
```

6.2 Windowed Band-Stop Filter Coefficients Implementation

```
import numpy as np

# sampling_rate, frequency ranges to block, desired stop-band attenuation, width of transition window
def getTaps(sampling_rate, stop_bands, A, transition_width):

    # Formula for num_taps of Kaiser window: (A - 8) / (2.285 * d_omega), where d_omega is the normalized transition width in radians
    num_taps = int((A - 8) / (2.285 * 2 * np.pi * (transition_width / sampling_rate)))

    if num_taps % 2 == 0:
        num_taps += 1 # Ensure the number of taps is odd so the filter order will be even

    # Define an ideal all-pass filter in the frequency domain
    H = np.ones(num_taps)

    # Apply the stop bands to the all-pass filter
    for low, high in stop_bands:
        # Normalize frequencies
        low = int(low * (num_taps / sampling_rate))
        high = int(high * (num_taps / sampling_rate))

        for i in range(low, high):
            H[i] = 0
            if i > 0: # ensure we don't try to mirror DC (0Hz)
                H[num_taps - i] = 0

    # Apply the IFFT to get the time-domain impulse response of the stop-band filter
    h = np.real(np.fft.ifft(H))

    # Shift Zero-frequency to center
    m = (num_taps - 1) // 2 # Define the centre index
    buffer = h.copy()
    split = num_taps - m
    h[0:m] = buffer[split:num_taps]
    h[m:num_taps] = buffer[0:split]

    n = np.arange(-m, m + 1) # Define a symmetric time-index array centred at zero for the kaiser function

    # Calculate beta for the Kaiser window
    if A > 50:
        beta = 0.1102 * (A - 8.7)
    elif 21 <= A <= 50:
        beta = 0.5842 * (A - 21) ** 0.4 + 0.07886 * (A - 21)
    else:
        beta = 0

    # Calculate the Kaiser Window function using a modified bessel function of the first kind
    # Kaiser window formula: I0 [beta * sqrt(1 - {(n - alpha) / alpha} ** 2)] / I0 * beta
    kaiser_window = np.i0(beta * np.sqrt(1 - (n / m) ** 2)) / np.i0(np.array([beta]))[0]

    # Window the stop-band filter with our kaiser window
    taps = h * kaiser_window

    return taps, num_taps
```

6.3 FIR Filter Implementation

```
import collections

class FIRfilter:
    def __init__(self, _coefficients):
        # Store the coefficients in this filter object (denoted by "self.")
        self.coefficients = _coefficients
        # Store the number of taps (== length of coefficients array)
        self.num_taps = len(self.coefficients)
        # Initialize a double-ended queue with a length of num_taps
        self.delay_line = collections.deque(maxlen=self.num_taps)

        return

    def dofilter(self, v):
        # Append incoming samples to the first index of the double-ended queue
        self.delay_line.appendleft(v)
        # initialize an accumulator for this window
        result = 0

        # Perform a convolution sum - multiply each sample in the buffer by its corresponding coefficient,
        # and add the result to the accumulator
        for i in range(len(self.delay_line)):
            result += self.delay_line[i] * self.coefficients[i]

        return result

    def dofilteradaptive(self, signal, noise, learningRate):
            """
            Adaptive filtering using the LMS algorithm.
            - signal: The input signal, to be filtered out
            - desired: The desired signal to be used for adaptation AKA ecg
            - mu: The step size for adaptation
            - result: The filtered signal
            """

            # perform a convolution sum to modify the amplitude and phase of the pure input signal
            filtered_signal = self.dofilter(signal)

            # calculate the error (the clean signal)
            error = noise - filtered_signal
            for i in range(len(self.delay_line)):
                # Update coefficients using LMS algorithm
                self.coefficients[i] += learningRate * error * self.delay_line[i]

            # Return the error signal (the difference between the desired signal and the filtered output)
            return error
```

6.4 Realtime Filtering Simulation Implementation

```
import fir_design as fir_design
import fir_filter as fir_filter
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

tsv = np.loadtxt("James_ecg_1.tsv", delimiter='\t')
ecg_signal = tsv[:,8].copy()

IMAGES = Path("images")
IMAGES.mkdir(parents = True, exist_ok = True)

# Performing FIR Filtering

num_samples = len(ecg_signal) # Number of Samples in ECG signal
sample_rate = 250 # Sampling rate of ECG signal (Hz)
width = 1 # desired width of the transition window
stop_Hz = [(0, 1), (45, sample_rate // 2)] # The cutoff frequency for the filter
A = 30 # stopband attenuation

# get the array of taps and number of taps (taps == coefficients)
taps, num_taps = fir_design.getTaps(sample_rate, stop_Hz, A, width)

# use FIRfilter to filter x with the FIR filter
filter = fir_filter.FIRfilter(taps)

# Simulate realtime filtering of the ECG
filtered_ecg_signal = np.zeros(num_samples)
for i in range(num_samples):
    filtered_ecg_signal[i] = filter.dofilter(ecg_signal[i])
```

```
measurement_time = num_samples / sample_rate
t = np.linspace(0 , measurement_time , num_samples)
```

6.5 Adaptive Filter Implementation

```
import fir_design as fir_design
import fir_filter as fir_filter
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

IMAGES = Path("images")
IMAGES.mkdir(parents = True, exist_ok = True)

tsv = np.loadtxt("James_ecg_1.tsv", delimiter='\t')
ecg_signal = tsv[:,8].copy()

# This data is not used for the adaptive filtering, but allows us to call getTaps to get the number of taps for the adaptive filter.
# The adaptive filter will be initialized with all coefficients set to zero.
num_samples = len(ecg_signal) # Number of Samples in ECG signal
sample_rate = 250 # Sampling rate of ECG signal (Hz)
width = 1 # desired width of the transition window
nyquist_freq = sample_rate // 2
stop_Hz = [(0, 1), (45, nyquist_freq)] # The cutoff frequency for the filter
A = 30 # stopband attenuation

# get the number of taps (taps == coefficients).
taps, num_taps = fir_design.getTaps(sample_rate, stop_Hz, A, width)

#Simulate realtime adaptive filtering of the ECG

# Initialize adaptive filter with num_taps coefficients all set to zero
adaptivefilter = fir_filter.FIRfilter(np.zeros(num_taps))

# Initialize an array to hold the adaptive filtered ECG signal
adaptive_filtered_ecg_signal = np.zeros(num_samples)

# Use 50Hz + DC shift as the signal for adaptive filtering (to be filtered out)
fiftyHz = np.sin(2 * np.pi * 50 / sample_rate * np.arange(num_samples)) + 0.3

# Use the filtered ECG signal as the noise reference for adaptive filtering
for i in range(num_samples):
    adaptive_filtered_ecg_signal[i] = adaptivefilter.dofilteradaptive(fiftyHz[i], ecg_signal[i], learningRate=0.001)

measurement_time = num_samples / sample_rate
t = np.linspace(0 , measurement_time , num_samples)
```

6.6 Heartbeat detection

```
import fir_design as fd
import fir_filter as ff
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

IMAGES = Path("images")
IMAGES.mkdir(parents = True, exist_ok = True)

tsv1 = np.loadtxt("James_ecg_1.tsv", delimiter='\t')
ecg_signal = tsv1[:,8].copy()

# Performing FIR Pre-Filtering to remove DC offset, baseline wander, and 50Hz mains interference

num_samples = len(ecg_signal) # Number of Samples in ECG signal
sample_rate = 250 # Sampling rate of ECG signal (Hz)
width = 1 # desired width of the transition window
stop_Hz = [(0, 3), (40, sample_rate // 2)] # The cutoff frequency for the filter
A = 30 # stopband attenuation

# get the array of taps and number of taps (taps == coefficients)
taps, num_taps = fd.getTaps(sample_rate, stop_Hz, A, width)

# use FIRfilter to filter x with the FIR filter
prefilter = ff.FIRfilter(taps)

# Simulate realtime filtering of the ECG
filtered_ecg_signal = np.zeros(num_samples)
for i in range(num_samples):
    filtered_ecg_signal[i] = prefilter.dofilter(ecg_signal[i])

tsv2 = np.loadtxt("Ewan_ecg_1.tsv", delimiter='\t')
```

```
ecg_template = tsv2[:,8].copy()

# use new prefilter to filter the ECG template signal. So delay line is empty
prefilter2 = ff.FIRfilter(taps)

# Simulate realtime filtering of the ECG template
filtered_ecg_template = np.zeros(len(ecg_template))
for i in range(len(ecg_template)):
    filtered_ecg_template[i] = prefilter2.dofilter(ecg_template[i])

#extract one PQRST complex from the filtered ECG template for matching
filtered_ecg_template = filtered_ecg_template[500:700]

# Reverse the filtered ECG template for cross-correlation
filtered_ecg_template = filtered_ecg_template[::-1]

matched_filter = ff.FIRfilter(filtered_ecg_template)

matched_filtered_ecg_signal = np.zeros(num_samples)

num_samples_template = len(matched_filtered_ecg_signal) # Number of Samples in ECG signal
measurement_time = num_samples_template / sample_rate
t2 = np.linspace(0 , measurement_time , num_samples_template)

#Realtime matched filtering of the filtered ECG signal with the filtered ECG template
last_peak_index = 0
current_peak_index = 0
heartrate = []
cooldown_period = 68  # 68/250Hz - limits heartrate to 220bpm
threshold = 3*10**-11

cooldown = 0
for i in range(num_samples):
    matched_filtered_ecg_signal[i] = matched_filter.dofilter(filtered_ecg_signal[i])

    # Square the matched filtered signal to enhance the peaks for easier detection of R-peaks
    matched_filtered_ecg_signal[i] = matched_filtered_ecg_signal[i] * matched_filtered_ecg_signal[i]

    if cooldown > 0:
        cooldown -= 1

    elif matched_filtered_ecg_signal[i] > threshold:
        last_peak_index = current_peak_index
        current_peak_index = i

        if last_peak_index > 0: # Ensure there are at least two beats before calculating heartrate
            heartrate.append(60 * sample_rate / (current_peak_index - last_peak_index))

        cooldown = cooldown_period
```